

openapi: 3.0.3

info:

title: PSP (Payment Service Provider) API

description: |

A comprehensive Payment Service Provider API that enables secure payment processing and user management for e-commerce platforms and webshops.

****User Management****

- User account registration and authentication for webshops
- Secure login and session management
- Comprehensive user profile management

****Payment Processing****

- Multi-channel payment processing supporting various payment methods
- Real-time transaction processing and validation
- Secure payment data handling with industry-standard encryption

****Subscription Management****

- Flexible subscription creation and management
- Support for multiple payment methods per subscription
- Renewal processes

****Administrative Features****

- Administrative access to user accounts and transaction data
- User management and monitoring tools

Supported Payment Methods

- ****Card Payments****:
Credit and debit card processing with secure tokenization
- ****QR Code Payments****:
Mobile-friendly QR code payment processing
- ****Cryptocurrency****:
Ethereum blockchain payment integration
- ****Digital Wallets****:
PayPal payment processing and management

version: 1.0.0

contact:

name: Team12

email: team12@gmail.com

servers:

- url: <https://api.psp.com/v1>
description: Production server (placeholder)
- url: <https://localhost:8090/v1>
description: local development

paths:

/api/auth/login:

post:

summary: User authentication

description: |

Authenticates user through Keycloak and returns access token and refresh token.

Uses Keycloak OAuth2 password grant flow to obtain tokens.

tags:

- Authentication

requestBody:

required: true

content:

application/json:

schema:

type: object

required:

- username

- password

properties:

username:

type: string

description: Username or email

example: "merchant@example.com"

password:

type: string

format: password

description: User password

example: "securePassword123"

responses:

'200':

description: Successful authentication

content:

application/json:

schema:

type: object

properties:

access_token:

type: string

description: JWT access token

example: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

refresh_token:

type: string

description: Refresh token for session renewal

example: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

expires_in:

type: integer

description: Access token expiration time in seconds

```
        example: 3600
        token_type:
          type: string
          example: "Bearer"
'401':
  description: Authentication failed
  content:
    application/json:
      schema:
        type: string
        example: "Keycloak login error: Invalid credentials"
'400':
  $ref: '#/components/responses/BadRequest'
```

/api/auth/register:

post:

summary: User registration

description: |

Registers a new user in the system by creating user in Keycloak.

Creates user account with provided credentials and personal information.

tags:

- Authentication

requestBody:

required: true

content:

application/json:

schema:

type: object

required:

- username
- email
- firstName
- lastName
- password

properties:

username:

type: string

description: Unique username

example: "merchant123"

email:

type: string

format: email

description: User email address

example: "merchant@example.com"

firstName:
 type: string
 description: User's first name
 example: "John"
lastName:
 type: string
 description: User's last name
 example: "Doe"
password:
 type: string
 format: password
 description: User password
 example: "securePassword123"

responses:

'201':
 description: Registration successful
 content:
 application/json:
 schema:
 type: string
 example: "Registration successful!"

'400':
 description: Registration failed - invalid data
 content:
 application/json:
 schema:
 type: string
 example: "Username already exists"

'500':
 description: Keycloak registration error
 content:
 application/json:
 schema:
 type: string
 example: "Keycloak registration error: Connection failed"

/api/auth/signout:

post:

summary: User logout

description: |

Logs out user by invalidating refresh token in Keycloak.

Requires refresh token to properly terminate the session.

tags:

- Authentication

requestBody:
 required: true
 content:
 application/json:
 schema:
 type: object
 required:
 - refreshToken
 properties:
 refreshToken:
 type: string
 description: Refresh token to invalidate
 example: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

responses:
 '200':
 description: Successfully signed out
 content:
 application/json:
 schema:
 type: string
 example: "You've been signed out!"
 '500':
 description: Logout error
 content:
 application/json:
 schema:
 type: string
 example: "Keycloak logout error: Connection failed"

/api/auth/send/code:

post:
 summary: Send verification code
 description: |
 Generates and sends 8-digit verification code to user's email.
 Code can be used for additional authentication steps.
 tags:
 - Authentication
 requestBody:
 required: true
 content:
 application/json:
 schema:
 type: object
 required:

- username

properties:

- username:
 - type: string
 - description: Username to send code to
 - example: "merchant@example.com"

responses:

'200':

description: Code sent successfully

content:

- application/json:
 - schema:
 - type: object
 - properties:
 - success:
 - type: boolean
 - example: true
 - message:
 - type: string
 - example: "OK"

'400':

\$ref: '#/components/responses/BadRequest'

/api/auth/refresh:

post:

summary: Refresh access token

description: |

Refreshes expired access token using valid refresh token.

Uses Keycloak refresh token grant to get new access token.

tags:

- Authentication

requestBody:

required: true

content:

- application/json:

- schema:

- type: object

- required:

- refreshToken

- properties:

- refreshToken:

- type: string

- description: Valid refresh token

- example: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

responses:

'200':

description: Token refreshed successfully

content:

application/json:

schema:

type: object

properties:

access_token:

type: string

description: New JWT access token

example: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

refresh_token:

type: string

description: New refresh token

example: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

expires_in:

type: integer

example: 3600

token_type:

type: string

example: "Bearer"

'500':

description: Refresh token error

content:

application/json:

schema:

type: string

example: "Keycloak refresh error: Invalid refresh token"

/api/checking_webshop_services:

post:

summary: Validate webshop and create checkout session

description: |

Validates webshop credentials and creates a new checkout session.

Checks if webshop details match records and has active subscriptions.

Returns redirect URL for payment processing.

tags:

- Checkout

parameters:

- name: Authorization

in: header

required: false

description: Bearer token for authentication

schema:

```
    type: string
    example: "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
requestBody:
  required: true
  content:
    application/json:
      schema:
        type: object
        required:
          - webShopId
          - webShopName
          - webShopUrl
          - userId
          - userFirstName
          - userLastName
          - userEmail
          - cart
        properties:
          webShopId:
            type: integer
            description: Unique identifier of the webshop
            example: 123
          webShopName:
            type: string
            description: Name of the webshop
            example: "My Online Store"
          webShopUrl:
            type: string
            format: uri
            description: URL of the webshop
            example: "https://mystore.com"
          userId:
            type: string
            description: Customer user ID
            example: "customer123"
          userFirstName:
            type: string
            description: Customer's first name
            example: "John"
          userLastName:
            type: string
            description: Customer's last name
            example: "Doe"
          userEmail:
```


type: string
format: email
description: Customer's email address
example: "john.doe@example.com"

cart:

type: array
description: Shopping cart items

items:

type: object

properties:

productId:

type: string

example: "prod123"

productName:

type: string

example: "Product Name"

quantity:

type: integer

example: 2

price:

type: number

format: float

example: 29.99

responses:

'200':

description: Webshop validated and session created successfully

content:

application/json:

schema:

type: string

description: Redirect URL for payment processing

example: "https://localhost:4201/available-service/session123?accessToken=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

'400':

description: Validation failed

content:

application/json:

schema:

type: string

examples:

invalid_webshop:

value: "The provided webshop details do not match our records. Please verify the webshop name and URL."

no_subscriptions:

value: "No active subscriptions found for this webshop."

'500':

\$ref: '#/components/responses/InternalServerError'

/actuator/health:

get:

summary: Health check endpoint

description: |

Returns the health status of the PSP service.

Used for monitoring, load balancer health checks, and service discovery.

tags:

- System

responses:

'200':

description: Service is healthy and operational

content:

application/json:

schema:

type: object

properties:

status:

type: string

description: Current health status

example: "UP"

timestamp:

type: string

format: date-time

description: Timestamp when health check was performed

example: "2024-01-15T10:30:00Z"

example:

status: "UP"

timestamp: "2024-01-15T10:30:00Z"

'500':

description: Service is unhealthy

content:

application/json:

schema:

type: object

properties:

status:

type: string

example: "DOWN"

timestamp:

type: string

format: date-time

error:

type: string

description: Error details if service is down

/api/user/active_payment_services:

get:

summary: Get active payment services

description: |

Retrieves list of currently active PSP payment services.

Filters services based on service discovery registry and returns only active ones.

Requires user authentication with appropriate role.

tags:

- PSP Services

security:

- bearerAuth: []

responses:

'200':

description: List of active payment services retrieved successfully

content:

application/json:

schema:

type: array

items:

type: object

properties:

id:

type: integer

description: Unique service identifier

example: 1

name:

type: string

description: Payment service name

example: "Credit Card Service"

description:

type: string

description: Service description

example: "Processes credit and debit card payments"

status:

type: string

enum: [active, inactive]

description: Current service status

example: "active"

serviceUrl:

type: string

format: uri

description: Service endpoint URL
example: "https://card-service.psp.com"
supportedCurrencies:
type: array
items:
type: string
description: List of supported currencies
example: ["RSD", "EUR", "USD"]
processingFee:
type: number
format: float
description: Processing fee percentage
example: 2.5

example:

- id: 1
name: "Credit Card Service"
description: "Processes credit and debit card payments"
status: "active"
serviceUrl: "https://card-service.psp.com"
supportedCurrencies: ["RSD", "EUR", "USD"]
processingFee: 2.5
- id: 2
name: "Bank Transfer Service"
description: "Direct bank transfer processing"
status: "active"
serviceUrl: "https://bank-service.psp.com"
supportedCurrencies: ["RSD", "EUR"]
processingFee: 1.0

'401':

\$ref: '#/components/responses/Unauthorized'

'403':

\$ref: '#/components/responses/Forbidden'

'500':

\$ref: '#/components/responses/InternalServerError'

/api/psp/requests/sendRequestQRCode/{sessionId}:

get:

summary: Generate QR code payment request

description: |

Processes QR code payment request for a specific session.

Validates API key through message queue, creates payment request,
and returns QR code payment data from the bank service.

tags:

- Requests

security:

```
- bearerAuth: []
parameters:
  - name: sessionId
    in: path
    required: true
    description: Unique identifier of the payment session
    schema:
      type: integer
      format: int64
      example: 12345
  - name: Authorization
    in: header
    required: true
    description: Bearer token for authentication
    schema:
      type: string
      example: "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
responses:
  '200':
    description: QR code payment request generated successfully
    content:
      application/json:
        schema:
          type: object
          properties:
            qrData:
              type: string
              description: QR code data for payment
              example: "bank1:payment:session12345:amount1000"
            paymentId:
              type: string
              description: Unique payment identifier
              example: "pay_qr_123456789"
            paymentUrl:
              type: string
              format: uri
              description: Payment processing URL
              example: "https://bank1.com/qr-payment/pay_qr_123456789"
            sessionId:
              type: integer
              description: Associated session ID
              example: 12345
            amount:
              type: number
```

format: float
description: Payment amount
example: 1000.00
currency:
type: string
description: Payment currency
example: "RSD"
expiresAt:
type: string
format: date-time
description: QR code expiration time
example: "2024-01-15T11:30:00Z"

'401':
\$ref: '#/components/responses/Unauthorized'
'403':
\$ref: '#/components/responses/Forbidden'
'400':
description: Invalid API key or session
content:
application/json:
schema:
type: string
example: "Invalid apiKey"
'500':
\$ref: '#/components/responses/InternalServerError'

/api/psp/requests/sendRequest/{sessionId}:

get:

summary: Create card payment request

description: |

Processes card payment request for a specific session.

Validates API key through message queue and creates payment request
with the bank service for card processing.

tags:

- Requests

security:

- bearerAuth: []

parameters:

- name: sessionId

in: path

required: true

description: Unique identifier of the payment session

schema:

type: integer

format: int64

example: 12345

responses:

'200':

description: Card payment request created successfully

content:

application/json:

schema:

type: object

properties:

paymentId:

type: string

description: Unique payment identifier

example: "pay_card_123456789"

paymentUrl:

type: string

format: uri

description: Payment processing URL

example: "https://bank1.com/card-payment/pay_card_123456789"

sessionId:

type: integer

description: Associated session ID

example: 12345

amount:

type: number

format: float

description: Payment amount

example: 1000.00

currency:

type: string

description: Payment currency

example: "RSD"

merchantId:

type: string

description: Merchant identifier

example: "MERCH_12345"

redirectUrl:

type: string

format: uri

description: URL to redirect after payment

example: "https://webshop.com/payment/success"

'401':

\$ref: '#/components/responses/Unauthorized'

'403':

\$ref: '#/components/responses/Forbidden'
 '400':
 description: Invalid API key or session
 content:
 application/json:
 schema:
 type: string
 example: "Invalid apiKey"
 '500':
 \$ref: '#/components/responses/InternalServerError'

/api/psp/requests/sendRequestCrypto:

get:
 summary: Get cryptocurrency wallet addresses
 description: |
 Retrieves available cryptocurrency wallet addresses for payment processing.
 Returns list of wallet IDs that can be used for crypto payments.
 tags:
 - Requests
 security:
 - bearerAuth: []
 responses:
 '200':
 description: Cryptocurrency wallet addresses retrieved successfully
 content:
 application/json:
 schema:
 type: array
 items:
 type: string
 description: List of available wallet addresses
 example: [
 "0x742d35cc36c6d008a6f3c8a29fbb5e0e04ee7c89",
 "0x8ba1f109551bd432803012645hac136c54c6d008",
 "0x1a2b3c4d5e6f7g8h9i0j1k2l3m4n5o6p7q8r9s0t"
]
 '401':
 \$ref: '#/components/responses/Unauthorized'
 '403':
 \$ref: '#/components/responses/Forbidden'
 '500':
 \$ref: '#/components/responses/InternalServerError'

/api/active_pspservices_bysession/{id}:

get:

summary: Get active PSP services for session

description: |

Retrieves active PSP services available for a specific session.

Returns services based on the webshop's active subscriptions.

Filters only currently active services.

tags:

- Sessions

parameters:

- name: id

- in: path

- required: true

- description: Unique identifier of the session

- schema:

- type: integer

- format: int64

- example: 12345

responses:

'200':

description: Active PSP services retrieved successfully

content:

application/json:

schema:

type: array

items:

type: object

properties:

id:

type: integer

description: Service identifier

example: 1

name:

type: string

description: PSP service name

example: "Credit Card Processing"

description:

type: string

description: Service description

example: "Visa and Mastercard payment processing"

active:

type: boolean

description: Service status

example: true

serviceType:

type: string

description: Type of payment service
enum: [card, bank_transfer, qr_code, crypto]
example: "card"

processingFee:
type: number
format: float
description: Processing fee percentage
example: 2.5

supportedCurrencies:
type: array
items:
type: string
description: Supported currencies
example: ["RSD", "EUR", "USD"]

example:

- id: 1
name: "Credit Card Processing"
description: "Visa and Mastercard payment processing"
active: true
serviceType: "card"
processingFee: 2.5
supportedCurrencies: ["RSD", "EUR", "USD"]
- id: 2
name: "QR Code Payment"
description: "NBS IPS QR code payments"
active: true
serviceType: "qr_code"
processingFee: 1.0
supportedCurrencies: ["RSD"]

'404':
description: Session not found
content:
application/json:
schema:
type: string
example: "Session not found"

'500':
description: Error retrieving session data
content:
application/json:
schema:
type: string
example: "Error retrieving session"

/api/session/{id}:

get:

summary: Get session details

description: |

Retrieves detailed information about a specific session including cart items and total price for payment processing.

tags:

- Sessions

parameters:

- name: id

in: path

required: true

description: Unique identifier of the session

schema:

type: integer

format: int64

example: 12345

responses:

'200':

description: Session details retrieved successfully

content:

application/json:

schema:

type: object

properties:

itemNames:

type: array

items:

type: string

description: List of item names in the cart

example: ["Product A", "Product B", "Product C"]

totalPrice:

type: number

format: float

description: Total price of all items in cart

example: 1250.00

example:

itemNames: ["Laptop", "Mouse", "Keyboard"]

totalPrice: 85000.00

'500':

description: Error retrieving session data

content:

application/json:

schema:

type: string
example: "Error retrieving session"

/api/user/create_subscription:

post:

summary: Create new subscription

description: |

Creates a new subscription for a user to a specific PSP service.

Validates that user doesn't already have an active subscription to the service.

Generates API key credentials for the subscription.

tags:

- Subscriptions

security:

- bearerAuth: []

requestBody:

required: true

content:

application/json:

schema:

type: object

required:

- serviceId

- userId

- subscriptionDuration

properties:

serviceId:

type: integer

description: ID of the PSP service to subscribe to

example: 1

userId:

type: integer

description: ID of the user creating subscription

example: 123

subscriptionDuration:

type: integer

description: Duration of subscription in months

minimum: 1

maximum: 36

example: 12

responses:

'201':

description: Subscription created successfully

content:

application/json:

schema:

type: object
properties:
 id:
 type: integer
 description: Subscription ID
 example: 456
 servId:
 type: integer
 description: PSP service ID
 example: 1
 userId:
 type: integer
 description: User ID
 example: 123
 isActive:
 type: boolean
 description: Subscription status
 example: true
 startDate:
 type: string
 format: date
 description: Subscription start date
 example: "2024-01-15"
 endDate:
 type: string
 format: date
 description: Subscription end date
 example: "2025-01-15"
 merchantId:
 type: string
 format: uuid
 description: Generated merchant ID
 example: "550e8400-e29b-41d4-a716-446655440000"
 merchantPassword:
 type: string
 description: Generated merchant password
 example: "mp_abc123def456"
 paymentServId:
 type: integer
 description: Payment service ID
 example: 1
 service:
 type: object
 properties:

```

      id:
        type: integer
        example: 1
      name:
        type: string
        example: "Credit Card Processing"
      description:
        type: string
        example: "Visa and Mastercard payment processing"
'400':
  description: Bad request - duplicate subscription or invalid data
  content:
    application/json:
      schema:
        type: string
        example: "You already have an active subscription to this payment service."
'404':
  description: Service or user not found
  content:
    application/json:
      schema:
        type: string
      examples:
        service_not_found:
          value: "Service not found"
        user_not_found:
          value: "User not found"
'401':
  $ref: '#/components/responses/Unauthorized'
'403':
  $ref: '#/components/responses/Forbidden'

```

/api/user_active_subscription/{userId}:

get:

summary: Get user's active subscriptions

description: |

Retrieves all active subscriptions for a specific user.

Returns only currently active subscriptions with their details.

tags:

- Subscriptions

security:

- bearerAuth: []

parameters:

- name: userId

type: string
example: "Credit Card Processing"
description:
type: string
example: "Visa and Mastercard payment processing"
active:
type: boolean
example: true
'401':
\$ref: '#/components/responses/Unauthorized'
'403':
\$ref: '#/components/responses/Forbidden'

/api/user_subscription/{userId}:

get:

summary: Get all user subscriptions (Admin only)

description: |

Retrieves all subscriptions (active and inactive) for a specific user.

Administrative endpoint with full subscription history access.

tags:

- Subscriptions

security:

- bearerAuth: []

parameters:

- name: userId

in: path

required: true

description: Unique identifier of the user

schema:

type: integer

format: int64

example: 123

responses:

'200':

description: All subscriptions retrieved successfully

content:

application/json:

schema:

type: array

items:

type: object

properties:

id:

type: integer

example: 456
serviceId:
 type: integer
 example: 1
userId:
 type: integer
 example: 123
isActive:
 type: boolean
 example: false
startDate:
 type: string
 format: date
 example: "2023-01-15"
endDate:
 type: string
 format: date
 example: "2024-01-15"
service:
 type: object
 properties:
 id:
 type: integer
 example: 1
 name:
 type: string
 example: "Credit Card Processing"

'401':
 \$ref: '#/components/responses/Unauthorized'
'403':
 \$ref: '#/components/responses/Forbidden'

/api/user/update_subscription:

put:

summary: Update subscription

description: |

 Updates an existing subscription with new details.

 Allows modification of subscription parameters and settings.

tags:

- Subscriptions

security:

- bearerAuth: []

requestBody:

 required: true

content:

application/json:

schema:

type: object

required:

- id

properties:

id:

type: integer

description: Subscription ID to update

example: 456

serviceId:

type: integer

description: PSP service ID

example: 1

userId:

type: integer

description: User ID

example: 123

isActive:

type: boolean

description: Subscription status

example: true

endDate:

type: string

format: date

description: New subscription end date

example: "2025-12-31"

responses:

'200':

description: Subscription updated successfully

content:

application/json:

schema:

type: string

example: "Subscription updated successfully"

'400':

description: Invalid subscription data

content:

application/json:

schema:

type: string

example: "Invalid subscription parameters"

'401':

```
    $ref: '#/components/responses/Unauthorized'
  '403':
    $ref: '#/components/responses/Forbidden'
/api/tests/public/hello:
  get:
    summary: Public test endpoint
    description: |
      Test endpoint that doesn't require authentication.
      Used for testing API connectivity and basic functionality.
    tags:
      - Testing
    responses:
      '200':
        description: Public endpoint accessed successfully
        content:
          text/plain:
            schema:
              type: string
              example: "Public endpoint – no auth required!"
```

```
/api/tests/user/hello:
  get:
    summary: User authentication test endpoint
    description: |
      Test endpoint that requires user authentication.
      Used for testing authentication mechanisms and user access.
    tags:
      - Testing
    security:
      - bearerAuth: []
    responses:
      '200':
        description: Authenticated user endpoint accessed successfully
        content:
          text/plain:
            schema:
              type: string
              example: "Hello, authenticated user!"
      '401':
        $ref: '#/components/responses/Unauthorized'
```

```
/api/tests/admin/hello:
  get:
    summary: Admin authorization test endpoint
```

description: |
Test endpoint that requires admin role authorization.
Used for testing role-based access control and admin permissions.

tags:
- Testing

security:
- bearerAuth: []

responses:
'200':
description: Admin endpoint accessed successfully
content:
text/plain:
schema:
type: string
example: "Hello, admin!"

'401':
\$ref: '#/components/responses/Unauthorized'

'403':
\$ref: '#/components/responses/Forbidden'

/api/user/{id}:

get:

summary: Get user information by ID

description: |
Retrieves detailed information about a specific user by their ID.
Requires user or admin role authorization.

tags:
- Users

security:
- bearerAuth: []

parameters:
- name: id
in: path
required: true
description: Unique identifier of the user
schema:
type: integer
format: int64
example: 123

responses:
'200':
description: User information retrieved successfully
content:
application/json:
schema:

type: object

properties:

id:

type: integer

description: User ID

example: 123

username:

type: string

description: Username

example: "merchant@example.com"

email:

type: string

format: email

description: User email address

example: "merchant@example.com"

firstName:

type: string

description: User's first name

example: "John"

lastName:

type: string

description: User's last name

example: "Doe"

companyName:

type: string

description: Company name (for business users)

example: "My Online Store Ltd."

webURL:

type: string

format: uri

description: Website URL (for webshop users)

example: "https://mystore.com"

role:

type: string

description: User role

enum: [ROLE_USER, ROLE_ADMIN, ROLE_BUSINESS_USER,
ROLE_PERSONAL_USER, ROLE_WEB_SHOP]

example: "ROLE_WEB_SHOP"

createdAt:

type: string

format: date-time

description: Account creation timestamp

example: "2024-01-15T10:30:00Z"

isActive:

```
        type: boolean
        description: Account active status
        example: true
'404':
  description: User not found
  content:
    application/json:
      schema:
        type: string
        example: "User not found with id: 123"
'401':
  $ref: '#/components/responses/Unauthorized'
'403':
  $ref: '#/components/responses/Forbidden'
```

/api/admin/users:

get:

summary: Get all users (Admin only)

description: |

Retrieves a list of all users in the system.

Administrative endpoint with access to complete user directory.

tags:

- Users

security:

- bearerAuth: []

responses:

```
'200':
  description: All users retrieved successfully
  content:
    application/json:
      schema:
        type: array
        items:
          type: object
          properties:
            id:
              type: integer
              example: 123
            username:
              type: string
              example: "merchant@example.com"
            email:
              type: string
              format: email
```

example: "merchant@example.com"

firstName:
type: string
example: "John"

lastName:
type: string
example: "Doe"

companyName:
type: string
example: "My Online Store Ltd."

webURL:
type: string
format: uri
example: "https://mystore.com"

role:
type: string
enum: [ROLE_USER, ROLE_ADMIN, ROLE_BUSINESS_USER,
ROLE_PERSONAL_USER, ROLE_WEB_SHOP]
example: "ROLE_WEB_SHOP"

createdAt:
type: string
format: date-time
example: "2024-01-15T10:30:00Z"

isActive:
type: boolean
example: true

example:

- id: 123
username: "merchant1@example.com"
email: "merchant1@example.com"
firstName: "John"
lastName: "Doe"
companyName: "Store One Ltd."
webURL: "https://store1.com"
role: "ROLE_WEB_SHOP"
createdAt: "2024-01-15T10:30:00Z"
isActive: true

- id: 124
username: "admin@psp.com"
email: "admin@psp.com"
firstName: "Admin"
lastName: "User"
role: "ROLE_ADMIN"
createdAt: "2024-01-10T09:00:00Z"

isActive: true

'401':

\$ref: '#/components/responses/Unauthorized'

'403':

\$ref: '#/components/responses/Forbidden'

components:

schemas:

CartDto:

type: object

properties:

items:

type: array

items:

\$ref: '#/components/schemas/CartItemDto'

description: List of items in the shopping cart

totalPrice:

type: number

format: double

description: Total price of all items in cart

example: 1250.00

totalItems:

type: integer

format: int64

description: Total number of items in cart

example: 3

example:

items:

- name: "Laptop"

price: 800.00

- name: "Mouse"

price: 25.00

totalPrice: 825.00

totalItems: 2

CartItemDto:

type: object

required:

- name

- price

properties:

name:

type: string

description: Name of the cart item

example: "Wireless Headphones"

price:
 type: number
 format: double
 description: Price of the item
 example: 99.99

CheckoutDataDto:

type: object

required:

- userId
- userFirstName
- userLastName
- userEmail
- cart
- webShopId
- webShopName
- webShopUrl

properties:

userId:

 type: integer
 format: int64
 description: Customer user identifier
 example: 123

userFirstName:

 type: string
 description: Customer's first name
 example: "John"

userLastName:

 type: string
 description: Customer's last name
 example: "Doe"

userEmail:

 type: string
 format: email
 description: Customer's email address
 example: "john.doe@example.com"

cart:

 \$ref: '#/components/schemas/CartDto'

webShopId:

 type: integer
 format: int64
 description: Webshop identifier
 example: 456

webShopName:

type: string
description: Name of the webshop
example: "My Online Store"
webShopUrl:
type: string
format: uri
description: URL of the webshop
example: "https://mystore.com"

CredentialDto:

type: object
required:
- username
- password
properties:
username:
type: string
description: Username or email for authentication
example: "merchant@example.com"
password:
type: string
format: password
description: User password
example: "securePassword123"

GenericResponse:

type: object
required:
- success
- message
properties:
success:
type: boolean
description: Indicates if operation was successful
example: true
message:
type: string
description: Response message
example: "Operation completed successfully"

PaymentDataDto:

type: object
properties:
paymentId:

type: integer
format: int64
description: Unique payment identifier
example: 789
paymentUrl:
type: string
format: uri
description: URL for payment processing
example: "https://payment.example.com/pay/789"

RegistrationDto:

type: object

required:

- firstName
- lastName
- email
- username
- password
- confirmPassword

properties:

firstName:

type: string
description: User's first name
example: "John"

lastName:

type: string
description: User's last name
example: "Doe"

email:

type: string
format: email
description: User's email address
example: "john.doe@example.com"

companyName:

type: string
description: Company name (optional)
example: "My Business Ltd."

websiteUrl:

type: string
format: uri
description: Company website URL (optional)
example: "https://mybusiness.com"

companyAddress:

type: string

description: Company address (optional)

example: "123 Business Street, City"

username:

type: string

description: Unique username

example: "johndoe"

password:

type: string

format: password

description: User password

example: "securePassword123"

confirmPassword:

type: string

format: password

description: Password confirmation

example: "securePassword123"

RequestDto:

type: object

required:

- merchantId
- merchantPassword
- amount
- merchantOrderId
- timestamp

properties:

merchantId:

type: string

format: uuid

description: Merchant unique identifier

example: "550e8400-e29b-41d4-a716-446655440000"

merchantPassword:

type: string

description: Merchant authentication password

example: "mp_abc123def456"

amount:

type: number

format: double

description: Payment amount

example: 1250.00

merchantOrderId:

type: integer

format: int64

description: Merchant's order identifier

example: 12345
timestamp:
 type: integer
 format: int64
 description: Request timestamp
 example: 1705320000
successUrl:
 type: string
 format: uri
 description: URL to redirect on successful payment
 example: "https://mystore.com/payment/success"
failedUrl:
 type: string
 format: uri
 description: URL to redirect on failed payment
 example: "https://mystore.com/payment/failed"
errorUrl:
 type: string
 format: uri
 description: URL to redirect on payment error
 example: "https://mystore.com/payment/error"

RequestPaymentDto:

 type: object
 properties:
 paymentId:
 type: integer
 format: int64
 description: Payment identifier
 example: 789
 paymentUrl:
 type: string
 format: uri
 description: Payment processing URL
 example: "https://bank1.com/payment/789"
 amount:
 type: number
 format: double
 description: Payment amount
 example: 1250.00
 successUrl:
 type: string
 format: uri
 description: Success redirect URL

example: "https://mystore.com/payment/success"
failedUrl:
type: string
format: uri
description: Failed redirect URL
example: "https://mystore.com/payment/failed"
errorUrl:
type: string
format: uri
description: Error redirect URL
example: "https://mystore.com/payment/error"

RequestQRCodePaymentDto:

type: object
properties:
qrData:
type: string
description: QR code data for payment
example: "bank1:payment:session12345:amount1250"
paymentId:
type: integer
format: int64
description: Payment identifier
example: 789
paymentUrl:
type: string
format: uri
description: Payment processing URL
example: "https://bank1.com/qr-payment/789"
amount:
type: number
format: double
description: Payment amount
example: 1250.00
successUrl:
type: string
format: uri
description: Success redirect URL
example: "https://mystore.com/payment/success"
failedUrl:
type: string
format: uri
description: Failed redirect URL
example: "https://mystore.com/payment/failed"

errorUrl:
 type: string
 format: uri
 description: Error redirect URL
 example: "https://mystore.com/payment/error"
qrPaymentId:
 type: string
 format: uuid
 description: QR payment unique identifier
 example: "550e8400-e29b-41d4-a716-446655440001"

SessionDto:
 type: object
 properties:
 itemNames:
 type: array
 items:
 type: string
 description: List of item names in session
 example: ["Laptop", "Mouse", "Keyboard"]
 totalPrice:
 type: number
 format: double
 description: Total price of session
 example: 1250.00

SubscriptionDto:
 type: object
 properties:
 id:
 type: integer
 format: int64
 description: Subscription identifier
 example: 123
 service:
 \$ref: '#/components/schemas/PSPService'
 userId:
 type: integer
 format: int64
 description: User identifier
 example: 456
 startDate:
 type: string
 format: date

description: Subscription start date

example: "2024-01-15"

endDate:

type: string

format: date

description: Subscription end date

example: "2025-01-15"

totalCost:

type: number

format: double

description: Total subscription cost

example: 299.99

isActive:

type: boolean

description: Subscription active status

example: true

subscriptionDuration:

type: integer

description: Duration in months

example: 12

merchantId:

type: string

format: uuid

description: Generated merchant ID

example: "550e8400-e29b-41d4-a716-446655440000"

merchantPassword:

type: string

description: Generated merchant password

example: "mp_abc123def456"

paymentServiceId:

type: integer

format: int64

description: Payment service identifier

example: 1

SubscriptionRequest:

type: object

required:

- userId
- serviceId
- subscriptionDuration

properties:

userId:

type: integer

format: int64
description: User identifier
example: 123
serviceId:
type: integer
format: int64
description: PSP service identifier
example: 1
subscriptionDuration:
type: integer
minimum: 1
maximum: 36
description: Subscription duration in months
example: 12

UserInfoDto:

type: object
properties:
id:
type: integer
format: int64
description: User identifier
example: 123
firstName:
type: string
description: User's first name
example: "John"
lastName:
type: string
description: User's last name
example: "Doe"
email:
type: string
format: email
description: User's email address
example: "john.doe@example.com"
companyName:
type: string
description: Company name
example: "My Business Ltd."
webURL:
type: string
format: uri
description: Company website URL

example: "https://mybusiness.com"
companyAddress:
type: string
description: Company address
example: "123 Business Street, City"

PSPService:

type: object

properties:

id:

type: integer

format: int64

description: Service identifier

example: 1

name:

type: string

description: Service name

example: "Credit Card Processing"

description:

type: string

description: Service description

example: "Visa and Mastercard payment processing"

active:

type: boolean

description: Service active status

example: true

serviceType:

type: string

enum: [card, bank_transfer, qr_code, crypto]

description: Type of payment service

example: "card"

processingFee:

type: number

format: double

description: Processing fee percentage

example: 2.5

supportedCurrencies:

type: array

items:

type: string

description: Supported currencies

example: ["RSD", "EUR", "USD"]

TokenRefreshRequest:

type: object

required:

- refreshToken

properties:

refreshToken:

type: string

description: Refresh token for token renewal

example: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."